🚀 Plano de Engenharia — Aplicação Fullstack de Gestão Orçamentária para Motorista de APP

1. Nome da Aplicação — Sugestões

Seguindo o critério de nomes curtos, memoráveis e com significado oculto (como "Uer"):

Nome	Significado oculto	Por quê funciona
Girô	"Giro" = fazer corridas (rodar) + "giro financeiro" (fluxo de caixa). O acento é identidade BR.	4 letras. Duplo sentido perfeito: **dirigir + dinheiro**. Ninguém nomeou um app assim.
Kursa	De "corrida" (kursa em sonoridade eslava) + "curso" (caminho/trajetória).	5 letras. Som internacional. Carrega a ideia de corrida e jornada.
Drin	Fusão de "**Dri**ver" + "di**n**heiro".	4 letras. Ultra minimalista. Fácil de falar e lembrar.

> TIP
>
> Minha recomendação é **Girô** — o nome carrega exatamente o que o app faz: controlar o *giro* (corridas e dinheiro) do motorista. É brasileiro, curto, único e com identidade visual forte (o acento no "ô" vira elemento gráfico do logo, como um volante ou moeda).

2. Stack Tecnológica Definida

Frontend (Web / PWA)

Tecnologia	Função
React 19 + **Vite**	Framework UI + bundler rápido
TypeScript (strict mode)	Tipagem estrita em todo o código
Tailwind CSS + **Shadcn/UI**	Design system utilitário + componentes acessíveis
Recharts	Gráficos de receita, combustível e despesas
TanStack Query (React Query)	Cache, sincronização e atualização em tempo real dos dados
Zustand	Estado global leve (sessão, tema, filtros ativos)
React Hook Form + **Zod**	Formulários com validação tipada
PWA (Workbox)	Instalável no celular, funciona offline

Backend (API REST)

Tecnologia	Função
Node.js 20+	Runtime do servidor
Fastify	Framework HTTP de alta performance
TypeScript (strict)	Tipagem end-to-end com o frontend
Prisma ORM	Abstração do banco + migrações versionadas
PostgreSQL 16	Banco de dados relacional principal
Zod	Validação de entrada (DTOs) no backend
JWT + **bcrypt**	Autenticação e hash de senhas
Helmet + **CORS** + **Rate Limiting**	Camada de segurança HTTP

3. Arquitetura de Módulos

```
giro/                          # Raiz do monorepo
├── docs/                      # Documentação e diagramas
│   ├── architecture.md
│   ├── er-diagram.md
│   └── api-contracts.md
├── packages/
│   ├── shared/                # Tipos e validações compartilhados (Zod schemas)
│   │   ├── schemas/           # Zod schemas usados por front e back
│   │   └── types/             # Interfaces TypeScript compartilhadas
│   │
│   ├── server/                # Backend (API)
│   │   └── src/
│   │       ├── core/
│   │       │   ├── config/         # Env vars, constants, timezone (America/Recife)
│   │       │   ├── database/
│   │       │   │   ├── migrations/ # Prisma migrations (DDL versionado)
│   │       │   │   ├── seeds/      # Categorias padrão, dados iniciais
│   │       │   │   └── client.ts   # Instância do PrismaClient (Singleton)
│   │       │   ├── errors/         # AppError, HttpError, mapeamento DB→HTTP
│   │       │   └── utils/          # Formatação BRL, cálculos de datas, helpers
│   │       ├── modules/
│   │       │   ├── auth/           # 🔐 Autenticação e sessão
│   │       │   │   ├── controllers/
│   │       │   │   ├── services/
│   │       │   │   ├── repositories/
│   │       │   │   └── types/
│   │       │   ├── earnings/       # 💰 Receita semanal (ganhos por corrida)
│   │       │   │   ├── controllers/
│   │       │   │   ├── services/
│   │       │   │   ├── repositories/
│   │       │   │   └── types/
│   │       │   ├── fuel-log/       # ⛽ Combustível, KM e abastecimentos
│   │       │   │   ├── controllers/
│   │       │   │   ├── services/
│   │       │   │   ├── repositories/
│   │       │   │   └── types/
│   │       │   ├── expenses/       # 📝 Despesas fixas e variáveis
│   │       │   │   ├── controllers/
│   │       │   │   ├── services/
│   │       │   │   ├── repositories/
│   │       │   │   └── types/
│   │       │   ├── vehicle/        # 🚗 Veículo (IPVA, licenciamento, manutenção)
│   │       │   │   ├── controllers/
```

```
│   │       │   │   ├── 📁 services/
│   │       │   │   ├── 📁 repositories/
│   │       │   │   └── 📁 types/
│   │       │   └── 📁 budgeting/      # 📊 Orçamento mensal, metas e consolidação
│   │       │       ├── 📁 controllers/
│   │       │       ├── 📁 services/
│   │       │       ├── 📁 repositories/
│   │       │       └── 📁 types/
│   │       ├── 📁 shared/
│   │       │   └── 📁 middlewares/    # Auth guard, rate limiter, error handler
│   │       └── 📄 main.ts            # Bootstrap do Fastify
│   │
│   └── 📁 web/                   # Frontend (React)
│       └── 📁 src/
│           ├── 📁 core/
│           │   ├── 📁 config/         # API base URL, feature flags
│           │   ├── 📁 theme/          # Tokens de cor, tipografia, dark mode
│           │   └── 📁 utils/          # Formatação de moeda, datas
│           ├── 📁 modules/
│           │   ├── 📁 auth/           # 🔐 Login, registro, proteção de rotas
│           │   │   ├── 📁 components/ # LoginForm, RegisterForm
│           │   │   ├── 📁 services/   # Chamadas à API de auth
│           │   │   ├── 📁 hooks/      # useAuth, useSession
│           │   │   └── 📁 types/
│           │   ├── 📁 dashboard/      # 📈 Painel principal com gráficos em tempo real
│           │   │   ├── 📁 components/ # WeeklyChart, FuelGauge, BalanceCard
│           │   │   ├── 📁 services/
│           │   │   ├── 📁 hooks/      # useDashboardData, useRealtimeUpdates
│           │   │   └── 📁 types/
│           │   ├── 📁 earnings/       # 💰 Registro e listagem de ganhos
│           │   │   ├── 📁 components/ # EarningForm, WeeklyEarningsTable
│           │   │   ├── 📁 services/
│           │   │   ├── 📁 hooks/
│           │   │   └── 📁 types/
│           │   ├── 📁 fuel-log/       # ⛽ Registro diário de KM + abastecimento
│           │   │   ├── 📁 components/ # FuelEntryForm, KmLogForm, FuelChart
│           │   │   ├── 📁 services/
│           │   │   ├── 📁 hooks/      # useFuelLog, useWeeklyFuelStats
│           │   │   └── 📁 types/
│           │   ├── 📁 expenses/       # 🧾 Gestão de despesas
│           │   │   ├── 📁 components/
│           │   │   ├── 📁 services/
│           │   │   ├── 📁 hooks/
│           │   │   └── 📁 types/
│           │   ├── 📁 vehicle/        # 🚗 Dados do veículo
│           │   │   ├── 📁 components/
```

```
│           │   │   ├── 📁 services/
│           │   │   ├── 📁 hooks/
│           │   │   └── 📁 types/
│           │   └── 📁 budgeting/      # 📊 Orçamento e metas
│           │       ├── 📁 components/
│           │       ├── 📁 services/
│           │       ├── 📁 hooks/
│           │       └── 📁 types/
│           ├── 📁 shared/
│           │   └── 📁 components/     # Button, Input, Modal, Card, CurrencyInput
│           └── 📄 main.tsx            # Entry point do React
└── 📄 package.json                   # Workspaces do monorepo
```

4. Modelagem de Dados Completa

5. Módulo de Combustível e KM — Detalhamento

Este é o módulo mais crítico para o dia a dia do motorista. Fluxo:

5.1 Registro Diário (DAILY_WORK_LOG)

```
┌──────────────────────────────────────────────┐
│  📅 Registro do Dia — 11/10/2026             │
├──────────────────────────────────────────────┤
│  KM Inicial:    [ 45.230  ]                  │
│  KM Final:      [ 45.412  ]                  │
│  Total KM:        182 km  (auto-calculado)   │
│                                              │
│  Ganho Bruto:   [ R$ 385,00 ]                │
│                                              │
│  ⛽ Abastecimentos do dia:                   │
│  ┌──────────────────────────────────────┐    │
│  │ Tipo: (●) Gasolina  (○) Etanol       │    │
│  │ Litros:          [ 25,5 ]            │    │
│  │ Preço/litro:     [ R$ 5,89 ]         │    │
│  │ Total:           R$ 150,20 (auto)    │    │
│  └──────────────────────────────────────┘    │
│  [+ Adicionar outro abastecimento]           │
│                                              │
│  Obs: [ Trânsito pesado na BR-101 ]          │
│                                              │
│  [💾 Salvar Registro]                        │
└──────────────────────────────────────────────┘
```

5.2 Fluxo de Dados em Tempo Real

Como funciona a atualização em tempo real:

1. O motorista salva um registro diário via formulário
2. O backend persiste e **recalcula automaticamente** os totais da semana corrente
3. A resposta da API retorna os novos totais semanais
4. O **React Query invalida o cache** das queries do dashboard
5. O gráfico **re-renderiza instantaneamente** com os dados atualizados
6. Não precisa de WebSocket — o `invalidateQueries` do TanStack Query já garante reatividade

5.3 Métricas Calculadas Automaticamente

Métrica	Fórmula	Exemplo
KM/L (Eficiência)	Total KM ÷ Litros consumidos	182 ÷ 25,5 = **7,14 km/L**
R$/KM (Custo por KM)	Gasto combustível ÷ Total KM	150,20 ÷ 182 = **R$ 0,83/km**
Lucro Líquido/KM	(Ganho - Combustível) ÷ KM	(385 - 150,20) ÷ 182 = **R$ 1,29/km**
% Combustível/Receita	Combustível ÷ Receita × 100	150,20 ÷ 385 × 100 = **39%**
Comparativo Gasolina vs Etanol	Custo/KM por tipo	Permite decidir qual compensa

6. Segurança — Camadas de Proteção

Camada	Implementação	Protege contra
Autenticação	JWT (access + refresh token) com httpOnly cookies	Acesso não autorizado
Hash de senha	bcrypt com salt rounds = 12	Vazamento de credenciais
Validação de entrada	Zod schemas em TODA rota (sem exceção)	Injection, dados malformados
Rate Limiting	Fastify rate-limit (100 req/min por IP)	DDoS, brute force
CORS	Whitelist de origens permitidas	Cross-origin attacks
Helmet	Headers de segurança HTTP	XSS, clickjacking
Sanitização	Trim + escape em strings antes de persistir	SQL injection, XSS stored
Soft Delete	Dados nunca são deletados fisicamente	Perda acidental de dados
Audit Trail	`created_at` e `updated_at` em todas as tabelas	Rastreabilidade

7. Fases de Desenvolvimento (Ciclo de Vida)

Fase 1 — Fundação (Semana 1-2)

- Inicialização do monorepo (workspaces npm/pnpm)
- Setup do backend (Fastify + TypeScript + Prisma)
- Setup do frontend (React + Vite + Tailwind + Shadcn)
- Modelagem do banco (Prisma schema) + migration inicial
- Seeds com categorias de despesas padrão
- Módulo `auth` (registro, login, JWT, refresh)
- Layout base do frontend (sidebar, header, rotas protegidas)

Fase 2 — Core do Negócio (Semana 3-4)

- Módulo `fuel-log` — Registro diário de KM + abastecimentos
- Módulo `earnings` — Registro e agregação de receita semanal
- Módulo `dashboard` — Gráficos de receita vs combustível (tempo real)
- Cálculos automáticos de eficiência (KM/L, R$/KM, etc.)

Fase 3 — Gestão Financeira (Semana 5-6)

- Módulo `expenses` — CRUD de despesas com categorias
- Módulo `vehicle` — IPVA, licenciamento, manutenção
- Módulo `budgeting` — Consolidação mensal, metas e saldo resultante
- Importação de dados da planilha Excel existente

Fase 4 — Polimento (Semana 7-8)

- Dark mode
- PWA (instalável no celular)
- Testes unitários nos services
- Testes de integração nas rotas
- Otimização de performance
- Deploy (Vercel/Railway/Render)

8. Gráficos do Dashboard (Visão Geral)

O dashboard principal terá os seguintes gráficos interativos:

1. 📊 **Receita vs Combustível (Barras empilhadas)** — por semana do mês corrente
2. 📈 **Evolução do Saldo Líquido (Linha)** — mês a mês
3. 🍩 **Distribuição de Despesas (Donut)** — por categoria
4. ⛽ **Eficiência de Combustível (Linha)** — KM/L por dia
5. 🔀 **Gasolina vs Etanol (Barras comparativas)** — custo/benefício por tipo
6. 💡 **Card de Insight** — "Hoje você lucrou R$ 1,29 por KM rodado"

> IMPORTANT
>
> Todos os gráficos atualizam em tempo real ao salvar um novo registro diário, sem necessidade de recarregar a página.

9. Fluxo de Dados Ponta a Ponta

Próximo passo

Ao aprovar este plano, iniciarei a **Fase 1** criando:

1. Estrutura do monorepo com workspaces
2. Schema Prisma completo com todas as entidades
3. Backend Fastify com módulo de autenticação
4. Frontend React com layout base e rotas